Common Subexpression Elimination (with Renaming)

The repeated statement is removed completely and every later use of its result is renamed to the earlier variable: t3 becomes t1, and then t5 becomes t4 (because t3 * t2 turns into t1 * t2, which t4 already computes). The program works on straight-line three-address code. A statement is removed only when both temporaries are assigned once, and an expression is reused only while its operands are unchanged.

Program (cse2.c)

```
#include <stdio.h>
#include <string.h>

#define MAX 20

typedef struct { char res[10], a[10], b[10], op; } Stmt;   /* res = a op b  (op = 0 -> res = a) */

Stmt code[MAX], out[MAX];                /* original code, optimized code */
Stmt avail[MAX];                         /* expressions already computed */
char from[MAX][10], to[MAX][10];          /* renames: from -> to */
int n, m, cnt, rcnt;

int assignments(char *name) {            /* how many times is 'name' assigned? */
    int c = 0;
    for (int i = 0; i < n; i++)
        if (strcmp(code[i].res, name) == 0) c++;
    return c;
}

void rename_var(char *s) {               /* replace an eliminated variable by the kept one */
    for (int i = 0; i < rcnt; i++)
        if (strcmp(from[i], s) == 0) { strcpy(s, to[i]); return; }
}

int find(Stmt *s) {                      /* is the same expression already computed? */
    for (int i = 0; i < cnt; i++)
        if (avail[i].op == s->op &&
            strcmp(avail[i].a, s->a) == 0 && strcmp(avail[i].b, s->b) == 0)
            return i;
    return -1;
}

void kill(char *name) {                  /* assignment to 'name' invalidates expressions using it */
    for (int i = 0; i < cnt; ) {
        if (strcmp(avail[i].res, name) == 0 ||
            strcmp(avail[i].a, name) == 0 || strcmp(avail[i].b, name) == 0) {
            for (int j = i; j < cnt - 1; j++) avail[j] = avail[j + 1];
            cnt--;
        } else i++;
    }
}

void show(Stmt *p, int k) {
    for (int i = 0; i < k; i++) {
        if (p[i].op) printf("  %s = %s %c %s\n", p[i].res, p[i].a, p[i].op, p[i].b);
        else         printf("  %s = %s\n", p[i].res, p[i].a);
    }
}

int main() {
    char line[100], c;
    int i, r, k;

    printf("Number of statements: ");
```

```
    fgets(line, sizeof(line), stdin);
    sscanf(line, "%d", &n);

    printf("Enter statements (like  t1 = a + b):\n");
    for (i = 0; i < n; i++) {
        fgets(line, sizeof(line), stdin);
        r = sscanf(line, "%9s = %9s %c %9s", code[i].res, code[i].a, &c, code[i].b);
        code[i].op = (r == 4) ? c : 0;
    }

    printf("\nOriginal code:\n");
    show(code, n);

    printf("\nEliminated:\n");
    for (i = 0; i < n; i++) {
        Stmt s = code[i];
        rename_var(s.a);                                /* use the kept variable names */
        if (s.op) rename_var(s.b);

        if (s.op) {
            k = find(&s);
            if (k >= 0 && assignments(s.res) == 1 && assignments(avail[k].res) == 1) {
                /* common subexpression: drop this statement, rename its result */
                printf("  %s = %s %c %s   (same as %s, so %s is renamed to %s)\n",
                       s.res, s.a, s.op, s.b, avail[k].res, s.res, avail[k].res);
                strcpy(from[rcnt], s.res);
                strcpy(to[rcnt], avail[k].res);
                rcnt++;
                continue;
            }
            kill(s.res);
            if (strcmp(s.res, s.a) != 0 && strcmp(s.res, s.b) != 0)
                avail[cnt++] = s;
        } else {
            kill(s.res);
        }
        out[m++] = s;
    }
    if (rcnt == 0) printf("  none\n");

    printf("\nAfter common subexpression elimination:\n");
    show(out, m);
    return 0;
}
```

How to run

```
gcc cse2.c -o cse2
./cse2
```

Sample input

```
6
t1 = a + b
t2 = c * d
t3 = a + b
t4 = t1 * t2
t5 = t3 * t2
t6 = t4 + t5
```

Output

```
Original code:                 After common subexpression elimination:
  t1 = a + b                     t1 = a + b
  t2 = c * d                     t2 = c * d
  t3 = a + b                     t4 = t1 * t2
  t4 = t1 * t2                   t6 = t4 + t4
  t5 = t3 * t2
  t6 = t4 + t5

Eliminated:
  t3 = a + b     (same as t1, so t3 is renamed to t1)
  t5 = t1 * t2   (same as t4, so t5 is renamed to t4)
```